

DCC Internet P2P Blockchain Chat

Alícia Marzola Chaves, José Gabriel Vieira de Souza

1. Descrição da Implementação

O sistema foi implementado em Python, utilizando a biblioteca padrão para comunicação de rede (socket), controle de concorrência (threading) e hashing (hashlib). A aplicação é dividida em três camadas principais:

1. **Interface de usuário (chat.py):** permite a entrada de comandos via terminal.
2. **Nó P2P (p2p_node.py):** gerencia conexões de rede, sincronização de histórico, mineração e disseminação.
3. **Mensagens e Histórico (chat_message.py e chat_history.py):** responsáveis pela estrutura e verificação dos dados armazenados.

A **troca de mensagens** entre os nós utiliza comunicação TCP via socket, com estrutura de mensagens definida conforme o enunciado:

- Todas as mensagens começam com um byte de identificação (msg_type) e seguem com campos codificados com struct usando network byte order (big endian).
- A função auxiliar `recv_all(conn, length)` garante a leitura completa do número exato de bytes esperados, evitando leituras parciais.

Os **tipos de mensagens implementados** são:

- **PeerRequest** [0x1]: enviado para solicitar a lista de pares.
- **PeerList** [0x2]: enviado em resposta, com lista de IPs dos pares conhecidos.
- **ArchiveRequest** [0x3]: não utilizado diretamente, pois os pares respondem automaticamente com **ArchiveResponse** ao se conectar.
- **ArchiveResponse** [0x4]: contém o histórico de mensagens do nó, enviado para novos pares ou em broadcast após mineração.

As mensagens são tratadas dentro da função `handle_peer()` que executa em uma thread separada para cada conexão, garantindo paralelismo no recebimento e envio de mensagens.

Identificação e Manutenção de Pares

Ao iniciar, o nó:

- Faz bind na porta 51511 e começa a escutar conexões.
- Se um peer conhecido for informado (via argumento), tenta se conectar a ele.

- Ao conectar-se a um peer, envia um PeerRequest, que retorna um PeerList.
- O nó se conecta recursivamente aos novos pares descobertos.

A lista de pares é armazenada em `self.peers` e é protegida por um `threading.Lock` para garantir acesso seguro entre threads. O sistema também roda uma thread periódica (`send_periodic_peer_requests`) que tenta reconectar pares inativos a cada 10 segundos.

Histórico de Chats e Verificação

O histórico de mensagens é gerenciado pela classe `ChatHistory`. Cada mensagem (`ChatMessage`) contém:

- `n`: número de caracteres.
- `text`: texto da mensagem em ASCII.
- `verifier`: código verificador de 16 bytes aleatórios.
- `md5`: hash MD5 de 16 bytes.

A serialização e deserialização são feitas com `struct` e respeitam o formato especificado. O método `ChatHistory.is_valid()` verifica recursivamente:

- Se o último hash começa com dois bytes zero.
- Se o último hash MD5 é correto (calculado sobre os últimos 20 chats, excluindo os 16 bytes do próprio hash).
- Se a sequência anterior também é válida (verificação recursiva).

A validação ocorre toda vez que um novo `ArchiveResponse` é recebido. O histórico local é atualizado apenas se o novo for válido e maior.

Mineração de Mensagens

A mineração é iniciada ao digitar uma mensagem no terminal. O método `mine_and_add_chat(msg)`:

1. Cria uma nova `ChatMessage` com o texto digitado.
2. Gera códigos verificadores aleatórios com `os.urandom(16)`.
3. Concatena os últimos 19 chats (ou menos) com o novo chat, serializa e remove os últimos 16 bytes (MD5).
4. Calcula o hash MD5 da sequência.
5. Verifica se o hash começa com 00 00.

Quando um hash válido é encontrado, a mensagem é adicionada ao histórico, e o novo histórico é disseminado para todos os pares via `broadcast_new_history()`.

Disseminação e Interface

A interface do usuário, via terminal, permite:

- `/history`: imprime o histórico local, com hash de cada mensagem.
- `/peers`: imprime os IPs conhecidos da rede.
- mensagens livres: dispara o processo de mineração e disseminação.

A disseminação do histórico ocorre com `socket.create_connection()` para cada peer, e o histórico completo serializado é enviado diretamente após a conexão.

2. Hash e Mineração

A biblioteca utilizada para o cálculo do hash MD5 foi a **hashlib**, que faz parte da biblioteca padrão da linguagem Python. Essa biblioteca fornece uma interface simples e eficiente para algoritmos de hashing como MD5, SHA-1, SHA-256, entre outros.

Na implementação, o cálculo do hash é feito por meio da chamada:

```
python hashlib.md5(data).digest()
```

Essa função recebe os dados em formato de bytes (``data``) e retorna o digest MD5 em 16 bytes.

Para cada nova mensagem, é criado um objeto da classe ``ChatMessage``. Esse objeto possui um campo chamado ``verifier``, que corresponde ao código de verificação de 16 bytes. Este campo é preenchido com uma sequência aleatória gerada por:

```
python os.urandom(16)
```

A cada iteração do processo de mineração, um novo código verificador é gerado dessa forma, garantindo aleatoriedade e imprevisibilidade.

A mineração consiste em encontrar um valor de ``verifier`` tal que o hash MD5 da sequência de mensagens satisfaça a seguinte condição:

- Os dois primeiros bytes do hash devem ser ``0x00 0x00``.

O procedimento é o seguinte:

1. São considerados os últimos 19 chats do histórico local (ou menos, se houver menos mensagens) e o novo chat que está sendo minerado.
2. As mensagens são serializadas com ``m.serialize()`` e concatenadas na ordem.

3. Os últimos 16 bytes do novo chat (que representam o próprio MD5 do novo chat) são removidos antes do cálculo do hash. Isso garante que o próprio hash não influencie no resultado, conforme especificado no enunciado.
4. O hash MD5 é calculado sobre os bytes restantes.
5. Se os dois primeiros bytes do hash forem iguais a `0x00 0x00`, a mineração é considerada bem-sucedida.

O código responsável por essa verificação é:

```
python if md5.startswith(b'\x00\x00'):
    new_chat.md5 = md5
    self.history.messages.append(new_chat)
    self.broadcast_new_history()
    break
```

O critério de parada é simples: o processo termina assim que um hash MD5 válido é encontrado, ou seja, com dois bytes iniciais iguais a zero.

3. Instruções para Execução

O programa foi implementado na linguagem **Python 3** e utiliza exclusivamente **bibliotecas da biblioteca padrão**. Dessa forma, não há necessidade de instalar dependências externas com ferramentas como pip. As bibliotecas utilizadas incluem socket para comunicação de rede, threading para execução concorrente, struct para empacotamento e desempacotamento de bytes, hashlib para cálculo de hash MD5 e os para geração de bytes aleatórios.

Para executar um nó do sistema, basta utilizar o **comando python3** seguido do nome do arquivo principal, que é **chat.py**, passando como primeiro argumento o **endereço IP local que o nó deve utilizar**. Opcionalmente, é possível informar como segundo argumento o **IP de um peer já conhecido que esteja ativo na rede**. Esse segundo argumento permite que o nó descubra outros peers e integre-se automaticamente à rede P2P. Por exemplo, para conectar diretamente à implementação de referência da matéria, o comando seria:

```
python3 chat.py 127.0.0.1 150.164.213.243
```

Nesse exemplo, o nó fará bind no IP local 127.0.0.1 e tentará se conectar ao peer conhecido no IP 150.164.213.243. Após a conexão, o nó requisita a lista de peers ativos e passa a se comunicar com todos eles. A partir desse ponto, o usuário poderá digitar mensagens no terminal, que serão mineradas e transmitidas automaticamente para a rede.

Para verificar se a disseminação de um chat foi bem-sucedida, basta observar os logs no terminal após o envio da mensagem. O sistema imprime mensagens informando o sucesso da mineração e, em seguida, a tentativa de envio do histórico para os pares conhecidos. Além

disso, é possível utilizar o comando /history dentro da interface interativa para visualizar o histórico local do nó, incluindo o texto das mensagens e seus respectivos hashes. Ao executar esse comando, o usuário deve conseguir visualizar a mensagem que foi inserida, o que confirma que ela foi adicionada corretamente ao histórico local.

Durante os testes com a implementação de referência da matéria, é possível verificar se a conexão foi estabelecida corretamente observando as mensagens de log. O sistema imprime claramente quando a conexão é aceita e quando mensagens do tipo PeerRequest, PeerList, ArchiveRequest e ArchiveResponse são recebidas. A presença desses logs indica que o protocolo de comunicação está sendo seguido corretamente e que o nó está interoperando com a rede global. Caso a mensagem minerada seja transmitida com sucesso, ela deverá eventualmente aparecer no histórico de outros nós, incluindo o servidor do professor, desde que eles considerem o histórico recebido como válido e mais recente.

Por fim, é recomendável executar o programa em um ambiente sem NAT, como a rede da UFMG, para garantir que os outros nós da rede consigam se conectar de volta ao seu programa.

4. Estado do Histórico

O histórico, quando enviada a mensagem, possuía 130 mensagens, e podemos conferir **apenas texto e hash** das últimas mensagens abaixo:

--- Mensagem 127 ---

Texto: Marcos Aguilar entrou no chat

Hash : 0000b8be18e1a058519f57360b9c5afd

--- Mensagem 128 ---

Texto: ola mundo

Hash : 0000de3c2a6b3d0b7b37ca206644eae9

--- Mensagem 129 ---

Texto: Group: [Nomes Completos dos Integrantes]

Hash : 0000e8770decfad11b0069b6f02d1a1d

--- Mensagem 130 ---

Texto: GALO

Hash : 0000c92bc1dee0d19fb5b22611e9f7c3

A última mensagem ‘GALO’ aparece, no histórico completo, da seguinte forma:

--- Mensagem 130 ---

Texto: GALO

Hash : 0000c92bc1dee0d19fb5b22611e9f7c3

Tamanho: 4

Mensagem ASCII: b'GALO'

Código de Verificação: b'\xafu\xef\x8dI>&\xa9`\x07\xb5\x17\xfl[t'
MD5: b'\x00\x00\xc9+\xc1\xde\xe0\xd1\x9f\xb5\xb2&\x11\xe9\xf7\xc3'

5. Conteúdo da Mensagem com Integrantes do Grupo

--- Mensagem 131 ---

Texto: Grupo: Alicia Marzola Chaves e Jose Gabriel Vieira de Souza

Hash : 00000810dd6a5bf8a47589836472b610

Tamanho: 59

Mensagem ASCII: b'Grupo: Alicia Marzola Chaves e Jose Gabriel Vieira de Souza'

Código de Verificação: b't\xd4u\xff0\r\xf8vk5\xbf\xe0\xe2^\xe2\x1f'

MD5: b'\x00\x00\x08\x10\xddj[\xf8\xa4u\x89\x83dr\xb6\x10'